

Roles y subredes

Momento 1: Antes de autenticarse

Cualquier dispositivo que se conecte a la red por cable, debe mandar una solicitud al servidor DHCP para que se le asigne una IP (debe ser un rango 192.168.100.0) ya que no solo puede haber un equipo nuevo, sino que pueden haber varios.

Momento 2: Después de autenticarse

Se crean un packet- in y se crean los flujos correspondientes de forma temporal para que permita solo tráfico hacia el portal (10.0.0.10).

Aquí se debe ingresar sus credenciales (Código PUCP y contraseña) y M1 debe validarlo y determinar su rol.

El DHCP asigna una IP del bloque CIDR correspondiente al rol. El estudiante por el momento se divide en 3 facultades (Ing. Telecomunicaciones 10.2.10.0/20 , Ing. Informática 10.2.20.0/20 y Ing. Electrónica 10.2.30.0/20)

Rol	Facultad	Subred origen	Ip destino permitida	Protocolo/Puerto	Acción	Justificación
Periodo antes de Autenticación						
Cuarentena (Temporal)	-	192.168.100.0/24	10.0.0.10 (portal)	TCP 80, 443	output: puerto Portal Cautivo	Estado temporal, cualquier dispositivo nuevo
Periodo después de Autenticación						
Visitante	-	10.1.0.0/24	Gateway internet	TCP 80, 443	output: Puerto Gateway	Acceso externo limitado
Estudiante	Telecomunicaciones	10.2.1.0/24	Servidores Cursos	TCP 80, 443	output: puerto Servidor 2	Máximo 100 PCs por facultad
	Informática	10.2.2.0/24	Servidores Cursos	TCP 80, 443	output: Servidor 3	Máximo 100 PCs por facultad
	Electrónica	10.2.3.0/24	Servidores Cursos	TCP 80, 443	output: Servidor 4	Máximo 100 PCs por facultad
Docente	-	10.3.0.0/24	Servidores Cursos + Notas	TCP 80, 443	output: Servidor 5	Personal académico
Admin TI	-	10.4.0.0/24	Cualquier destino	Cualquiera	output: destino	Personal técnico, acceso total

Lista de Servidores

Servidor 1: Autenticación (Portal Cautivo)
 Servidor 2: Facultad de Telecomunicaciones (Cursos)
 Servidor 3: Facultad de Informática (Cursos)
 Servidor 4: Facultad de Electrónica (Cursos)
 Servidor 5 Notas (Docente)
 Servidor 6: Gestión de red/ Panel de control (Admin)
 Gateway (Visitante)
 Servidor DHCP 10.0.0.2

Servidor	IP	TCP	Quien accede
Portal Cautivo	10.0.0.10	80, 443	192.168.100.0/24
Cursos Telecom	10.0.0.21	80, 443	10.2.1.0/24 y 10.3.0.0/24
Cursos Informática	10.0.0.22	80, 443	10.2.2.0/24 y 10.3.0.0/24
Cursos Electrónica	10.0.0.23	80, 443	10.2.3.0/24 y 10.3.0.0/24
Notas	10.0.0.30	80, 443	10.3.0.0/24
Panel Admin Ti	10.0.0.40	80, 443	10.4.0.0/24
Gateway	192.168.201.1	-	10.1.0.0/24, 10.2.1.0/24, 10.2.2.0/24, 10.2.3.0/24 y 10.3.0.0/24

De form general y ya teniendo definido los roles y servidores correspondiente, se tendría la siguiente relación:

Rol	Ip origen	Ip destino	Protocolo / Puerto	Ip destino bloqueado	Justificación
Visitante	10.1.0.0/24	192.168.201.1 (Gateway)	TCP 80, 443	10.0.0.21 10.0.0.22 10.0.0.23 10.0.0.30 10.0.0.40	No tiene acceso a Cursos y Notas
Estudiante Telecom	10.2.1.0/24	10.0.0.21	TCP 80, 443	10.0.0.22 10.0.0.23	Tiene acceso a su

				10.0.0.30 10.0.0.40	Facultad, sin notas ni Admin
Estudiante Informatica	10.2.2.0/24	10.0.0.22	TCP 80, 443	10.0.0.21 10.0.0.23 10.0.0.30 10.0.0.40	Tiene acceso a su Facultad, sin notas ni Admin
Estudiante Electrónica	10.2.3.0/24	10.0.0.23	TCP 80, 443	10.0.0.21 10.0.0.22 10.0.0.30 10.0.0.40	Tiene acceso a su Facultad, sin notas ni Admin
Docente	10.3.0.0/24	10.0.0.21, 10.0.0.22, 10.0.0.23, 10.0.0.30	TCP 80, 443	10.0.0.40/24	No tiene acceso a infraestructur a SDN

Definición de las DBs (credenciales y sesiones)

1. Esquema 1: tablas de credenciales y roles

SCHEMA:Auth

Tabla Usuarios (Estudiante, Docente, Admin TI): almacena las credenciales de los usuarios

Campo	Tipo	Restricción	Descripción
id_usuario	in	PK	identificación único autogenerado
codigo_pucp	VARCHAR(20)	No nul	código PUCP del estudiante/Docente
password_hash	VARCHAR(255)	No null	contraseña hasheada
estado_cuenta	VARCHAR(20)	Default 'Activo'	Valores: ACTIVO, INACTIVO , BLOQUEADO

En caso sea visitante, no tiene código ni contraseña para el Portal Cautivo. Se añade un botón para una sesión temporal con limitaciones básicas.

[Aun me falta definir la logica para Visitante :’v]

Tabla Roles: define los roles existentes y su bloque CIDR al que le corresponda

Campo	Tipo	Restriccion	Descripcion
id_rol	SERIAL	PK	identificador unico del rol
nombre_rol	VARCHAR(50)	no null	Estudiante (teleco, informatica, electronica), Docente, Admin TI, Visitante
cidr	VARCHAR(50)	no null	las Ip de los roles

id_rol	nombre_rol	cidr
1	Visitante	10.1.0.0/24
2	Estudiante_Telecom	10.2.1.0/24
3	Estudiante_Informatica	10.2.2.0/24
4	Estudiante_Electronica	10.2.3.0/24
5	Docente	10.3.0.0/24
6	Admin_TI	10.4.0.0/24

2. Esquema 2: tablas de sesiones activas

Campo	Tipo de dato	Restricciones / Descripción
id_sesion	SERIAL	PK. Identificador único de la sesión
id_usuario	INT	No nulo. FK hacia auth.usuarios(id_usuario)
mac_address	VARCHAR(17)	No nulo. Dirección MAC del dispositivo. UNIQUE (una MAC solo puede tener una sesión)
ip_asignada	VARCHAR(15)	No nulo. Dirección IP asignada al usuario
cidr_rol	VARCHAR(18)	No nulo. Rango CIDR asignado según el rol del usuario
switch_dpuid	VARCHAR(30)	No nulo. Identificador del switch (DPID en SDN/OpenFlow)
in_port	INT	No nulo. Puerto físico del switch donde se conecta el dispositivo
login_timestamp	TIMESTAMP	Por defecto CURRENT_TIMESTAMP. Momento de

		inicio de sesión
estado	VARCHAR(20)	Por defecto 'ACTIVA'. Valores permitidos: 'ACTIVA', 'CERRADA', 'REVOCADA'

Aquí el Estado de la sesión tendrá 3 valores: ACTIVA, CERRADA, REVOCADA.

- CERRADA: indica cuando el usuario cerró sesión
- REVOCADA: indica que M4 mandó a terminar la sesión por actividad sospechosa

Tabla: lista_negra (referencia para cuando M4 revoque una sesión activa)

Campo	Tipo de dato	Restricciones / Descripción
id_amenaza	SERIAL	PK. Identificador único autoincremental de la amenaza
identificador	VARCHAR(20)	No nulo. Puede ser una dirección IP o MAC
tipo_identificador	VARCHAR(10)	No nulo. Solo permite valores: 'IP' o 'MAC' (CHECK constraint)
motivo_bloqueo	TEXT	No nulo. Describe la razón del bloqueo (no se incluirlo)
fecha_deteccion	TIMESTAMP	Por defecto CURRENT_TIMESTAMP. Fecha en que se detectó la amenaza
expira_bloqueo	TIMESTAMP	Opcional (NULL permitido). Indica cuándo deja de aplicarse el bloqueo

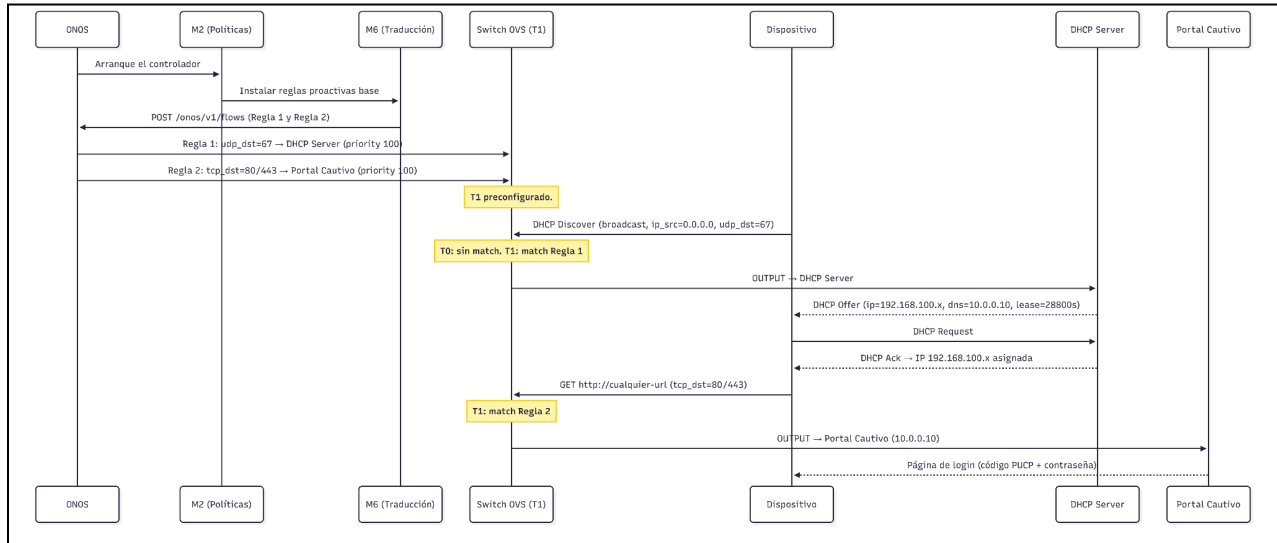
Flujo de Autenticación (Estudiantes, Docentes)

Caso 1: Inicialización con DHCP (Proactivo)

Cuando un dispositivo se conecta por cable, el sistema ya tiene reglas preinstaladas que permiten obtener IP y acceder al portal cautivo

- cuando arranca ONOS, M2 instruye a M6 a instalar de forma proactiva 2 reglas:
 - regla 1: DHCP permitido (udp_dst= 67, ip_dst=255.255.255.255 y output: puerto DHCP server 10.0.0.2)
 - regla 2: redirección al Portal Cautivo (tcp_dst= 80/443 y output: puerto Portal Cautivo 10.0.0.10)
- el dispositivo se conecta por primera vez y envía un DHCP Discover en broadcast porque no tiene una IP asignada
- el switch recibe el paquete y revisa las tablas del pipeline (T1: aplica regla 1)
- el DHCP Discover llega al servidor DHCP y este responde con un DHCP Offer asignando una IP de la subred 'cuarentena'

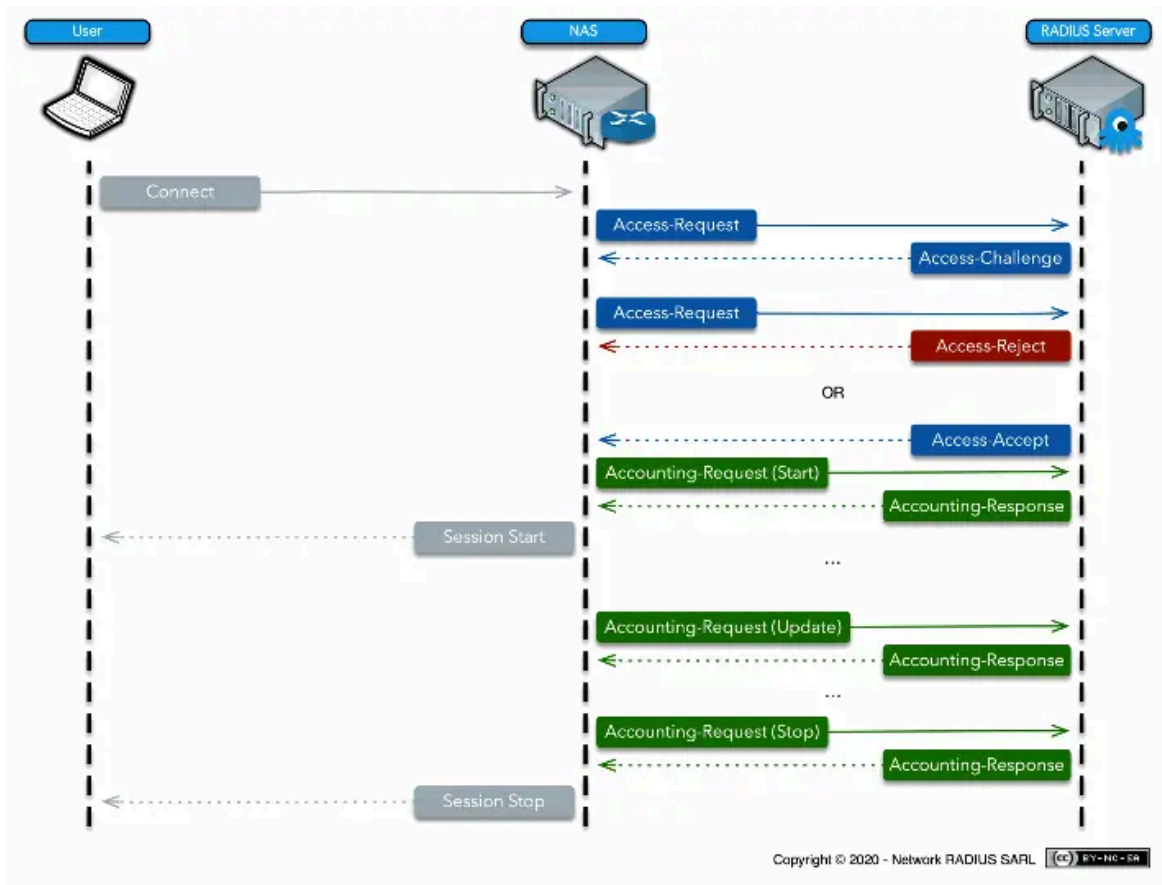
5. el dispositivo responde con DHCP request aceptando la IP ofrecida y el servidor confirma con DHCP Ack. El dispositivo ya tendría su IP configurado
6. El usuario que accede a ese dispositivo ya puede acceder al Portal Cautivo por la regla 2 del T1.
7. El portal cautivo responde mostrando la pagina login en donde se debe ingresar el usuario con su código PUCP y contraseña.



Flujo de asignación de IP temporal

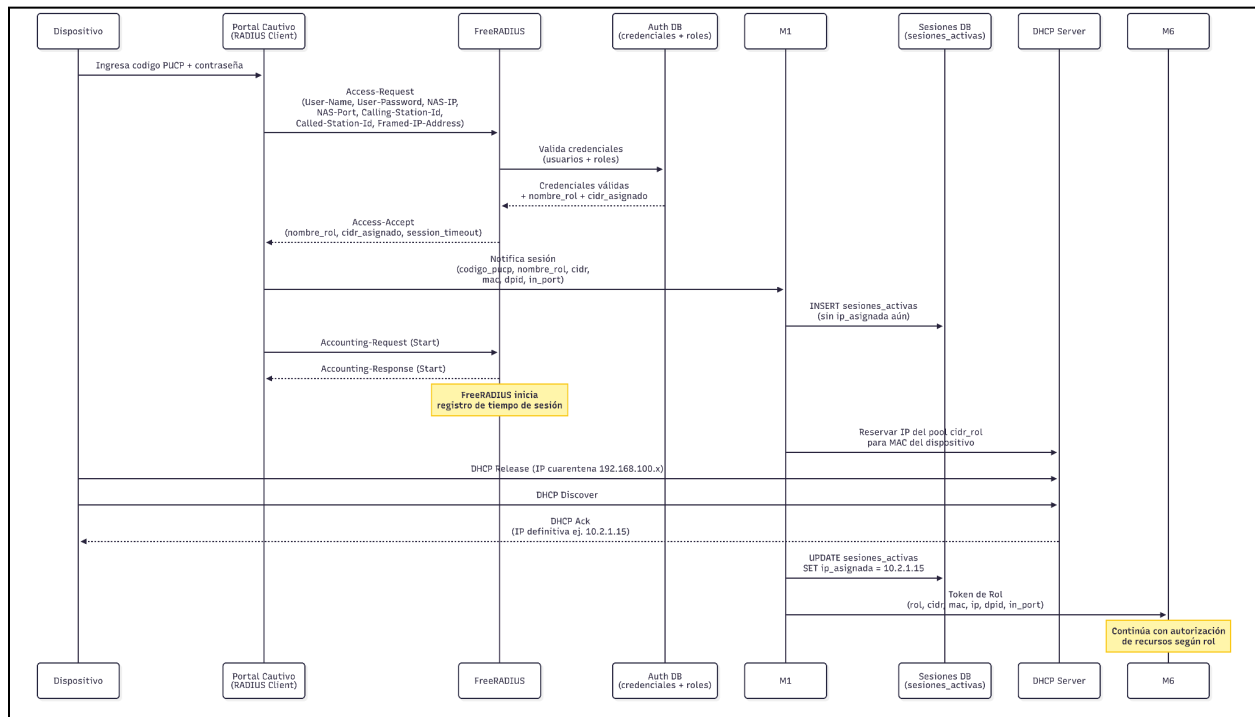
Caso 2: Autenticación (proactivo)

El dispositivo ya tiene configurado la IP y tiene acceso al Portal Cautivo, entonces, ahora el usuario debe ingresar sus credenciales y FreeRADIUS se encarga de validar (match o no) las credenciales y dar su rol asignado.



1. El usuario ingresa sus código PUCP y contraseña en el formulario del portal cautivo (RADIUS client) y este mismo genera un Access request hacia FreeRADIUS con el formato:
 - 1.1. **Atributos Access Request:**
 - 1.1.1. **User-Name:** Código PUCP
 - 1.1.2. **User-Password:** contraseña
 - 1.1.3. **NAS-IP-Address:** 10.0.0.10
 - 1.1.4. **NAS-Port:** Puerto físico del switch
 - 1.1.5. **Calling-Station-Id:** MAC del dispositivo
 - 1.1.6. **Called-Station-Id:** DPID del switch
 - 1.1.7. **Framed-IP-Address:** IP del dispositivo (192.168.100.0)
2. FreeRADIUS recibe el Access-Request y consulta el esquema auth de PostgreSQL para validar las credenciales:
 - 2.1. **Credenciales inválidas:** FreeRADIUS responde con Access-Reject + Reply-Message: Credenciales inválidas. El portal cautivo muestra el mensaje de error al usuario. Fin del flujo.
 - 2.2. **Credenciales válidas:** FreeRADIUS responde con Access accept + (nombre del rol, su CIDR asignado y tiempo de session)

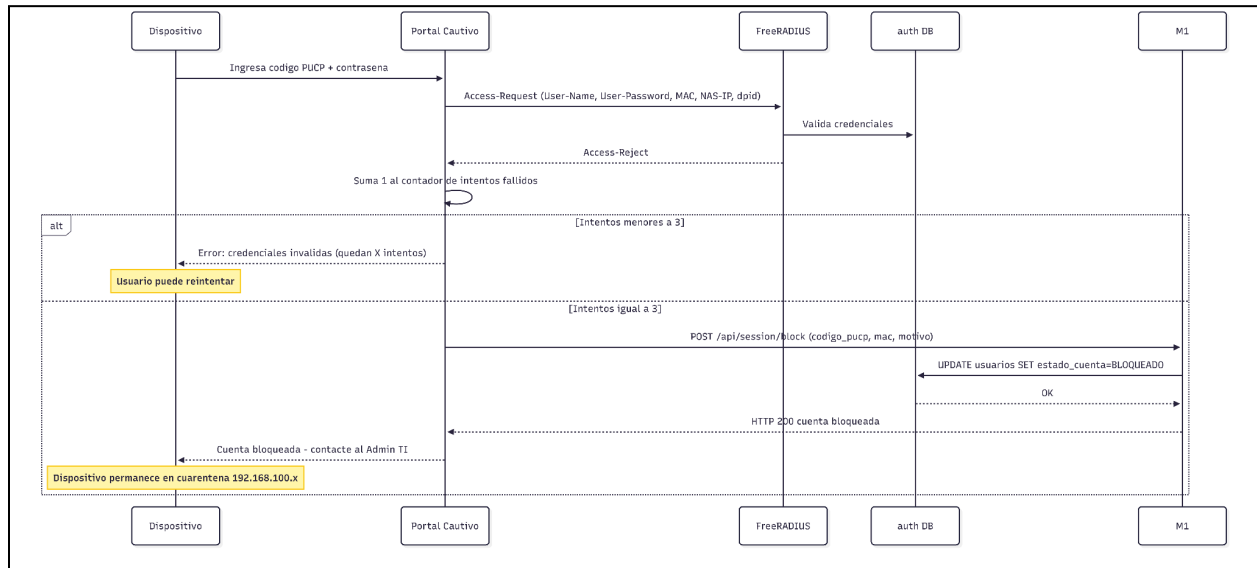
- El portal cautivo recibe el Access-Accept y notifica a M1 con todos los datos de la sesión: **código PUCP, nombre del rol, CIDR asignada, MAC del dispositivo, DPID del switch y su puerto de conexión.**
- M1 registrar la sesión creada en su tabla de sesiones activas con los parámetros mencionados anteriormente
- El portal cautivo envía Accounting-Request (Start) a FreeRADIUS para registrar el inicio de la sesión. FreeRADIUS responde con Accounting-Response (Start). Desde este momento FreeRADIUS lleva el registro de tiempo de sesión del usuario.
- M1 instruya al servidor DHCP a reservar una IP del pool 10.2.1.0/24 (ejemplo sea estudiante teleco) para la MAC. El dispositivo realiza un DHCP release a su IP cuarentena y realiza de nuevo su proceso de discover para obtener una nueva IP con su subred asignada para su rol.
- M1 actualiza la sesión activa con la nueva IP asignada y emite el Token de Rol hacia M6 para que continúe con la autorización de recursos de acuerdo a su rol.



Flujo para credenciales válidas

- sigue el mismo flujo hasta el paso 2 cuando el portal cautivo recibe un Access-Reject y suma 1 al contador de intentos fallidos (máximo 3) para ese codigo_pucp + MAC.
 - Intento menor a 3: muestra un mensaje de error y permite reintentar.
 - intento igual a 3: el portal notifica a M1 via POST /api/session/block con {codigo_pucp, mac_address, motivo: "3 intentos fallidos"}.
- M1 recibe la notificación y actualiza el estado de la cuenta en Autenticación.usuario

3. El portal muestra el mensaje "Cuenta bloqueada. Contacte al Administrador TI." El dispositivo permanece en cuarentena con IP 192.168.100.x. No se crea ninguna sesión en sessions.sesiones_activas.



Flujo para credenciales inválidas

Flujo de autenticación (Invitado)

NO HAY ROL VISITANTE 😊

Fuentes RADIUS:

<https://www.iana.org/assignments/radius-types/radius-types.xhtml#radius-types-2>

https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/sec_usr_radatt/configuration/xr-16/sec-usr-radatt-xr-16-book/sec-rad-ov-ietf-attr.html

Políticas y Autorización:

M2 es el módulo encargado de determinar las acciones que puede realizar el usuario autenticado en la red (Policy Decision Point) basándose en su rol y las envía a M6 para que este las traduzca y envíe al Controller que se encargará de instalarlas en los switches .

Tabla RBAC:

Los permisos son organizados de acuerdo al rol y los recursos que este desea acceder, principalmente el permiso de HTTP/HTTPS (TCP 80/443) desde la subred del rol hacia la IP del servidor donde se encuentra el recurso.

Rol	Portal cautivo (10.0.0.10)	Cursos Telecom (10.0.0.21)	Cursos Informática (10.0.0.22)	Cursos Electrónica (10.0.0.23)	Servidor Notas (10.0.0.30)	Panel Admin TI (10.0.0.40)	Gateway Internet (192.168.201.1)
Cuarentena	TCP 80/443						
Est. Teleco (10.2.1.0/24)		TCP 80/443					TCP 80/443
Est. Informática			TCP 80/443				TCP 80/443

(10.2.2.0/24)							
Est. Electrónica (10.2.3.0/24)				TCP 80/443			TCP 80/443
Docente (10.3.0.0/24)		TCP 80/443	TCP 80/443	TCP 80/443	TCP 80/443		TCP 80/443
Admin TI (10.4.0.0/24)							

Rol	IP Origen	Servidor	IP Destino	Puerto	Acción	Tabla	Motivo
Cuarentena	192.168.100.0/24	Portal cautivo	10.0.0.10	80, 443	ALLOW	T1 (proactivo)	Proceso de validación previa
		DHCP	255.255.255.255	UDP 67	ALLOW	T1 (proactivo)	Asignación de IP en la red
		Otro	-	-	DROP	T1	Aislamiento
Estudiante Telecom	10.2.1.0/24	Cursos Telecom	10.0.0.21	80, 443	ALLOW	T2	Acceso a cursos de su facultad
		Cursos Informática	10.0.0.22	80, 443	DENY	T3	Acceso a cursos de otra facultad
		Cursos Electrónica	10.0.0.23	80, 443	DENY	T3	Acceso a cursos de otra facultad
		Portal de Notas	10.0.0.30	80, 443	DENY	T3	Acceso a recurso no autorizado
		Panel de Admin	10.0.0.40	80, 443	DENY	T3	Acceso a recurso no autorizado
		Cursos Telecom	10.0.0.21	80, 443	DENY	T3	Acceso a cursos de otra

Estudiante Informática	10.2.2.0/24						facultad
		Cursos Informática	10.0.0.22	80, 443	ALLOW	T2	Acceso a cursos de su facultad
		Cursos Electrónica	10.0.0.23	80, 443	DENY	T3	Acceso a cursos de otra facultad
		Portal de Notas / Panel de Admin	10.0.0.30 10.0.0.40	80, 443	DENY	T3	Acceso a recurso no autorizado
Estudiante Electrónica	10.2.3.0/24	Cursos Telecom	10.0.0.21	80, 443	DENY	T3	Acceso a cursos de otra facultad
		Cursos Informática	10.0.0.22	80, 443	DENY	T3	Acceso a cursos de otra facultad
		Cursos Electrónica	10.0.0.23	80, 443	ALLOW	T2	Acceso a cursos de su facultad
		Portal de Notas / Panel de Admin	10.0.0.30 10.0.0.40	80, 443	DENY	T3	Acceso a recurso no autorizado
Docente	10.3.0.0/24	Cursos Telecom	10.0.0.21	80, 443	ALLOW	T2	Acceso a cursos
		Cursos Informática	10.0.0.22	80, 443	ALLOW	T2	Acceso a cursos
		Cursos Electrónica	10.0.0.23	80, 443	ALLOW	T2	Acceso a cursos
		Portal de Notas	10.0.0.30	80, 443	ALLOW	T2	Edición e ingreso de notas
		Panel de Admin	10.0.0.40	80, 443	DENY	T3	Acceso a recurso no autorizado

Admin TI	10.4.0.0/24	Servidores de cursos y notas	10.0.0.21 10.0.0.22 10.0.0.23 10.0.0.30	80, 443	ALLOW	T2	Acceso a todos los servidores
		Panel de admin	10.0.0.40	80, 443	ALLOW	T2	Manejo de la infraestructura

Como se observa en la tabla los roles no poseen acceso al portal, ya que se considera que se encuentran autenticados una vez que salieron de cuarentena; asimismo, se les restringe el acceso a los diferentes servidores de acuerdo a sus necesidades en la plataforma a excepción del Admin que tiene acceso completo desde cualquier destino hacia cualquier puerto TCP/UDP (SSH, HTTP, HTTPS, gestión, etc).

OPA como microservicio de M2:

El motor de políticas empleado para M2 será OPA (Open Policy Agent), este se correrá como un microservicio fuera del código del mismo. La independencia de OPA permite que ante fallos sea posible actualizarlo o reiniciarlo sin afectar el funcionamiento del sistema; además, al tener su propia memoria y procesador cuando se da el caso de múltiples consultas OPA puede caer pero no interfiere en el accionar de M6.

Esquema BD de políticas:

El módulo M2 se basa en Open Policy Agent (OPA) donde se expone un endpoint HTTP que recibirá un input que contiene usuario, rol, VLAN, IP origen y punto de conexión. Las políticas RBAC definidas antes son almacenadas en la tabla de políticas en la DB de PostgreSQL, al inicializarse el microservicio OPA este se conecta a PostgreSQL y realiza una carga de data masiva que se usará como data.

- Tabla de recursos:

Campo	Tipo	Restricción	Descripción
id_recurso	INT	PK	Identificador único
nombre_recurso	VARCHAR (50)	NOT NULL	Recurso: curso_teleco, notas, panel_admin
ip_dst	VARCHAR (18)	NOT NULL	IP destino del servidor
puerto	INT	NOT NULL	Puerto TCP

protocolo	VARCHAR (5)	NOT NULL	TCP o UDP
-----------	-------------	----------	-----------

- Tabla de permisos según rol (RBAC):

Campo	Tipo	Restricción	Descripción
id_permiso	INT	PK	Identificador único
id_rol	INT	FK (roles)	Rol evaluado
id_recurso	INT	FK (recursos)	Recurso evaluado
accion	VARCHAR(10)	NOT NULL	allow o deny
tabla	VARCHAR(5)	NOT NULL	T2 (CIDR) O T3 (usuario)
prioridad	INT	NOT NULL	Prioridad de la regla OpenFlow
timeout	INT	NULL	Permanencia de la regla
activo	BOOLEAN	TRUE	Uso de la regla

- Tabla de políticas temporales:

Campo	Tipo	Restricción	Descripción
id	INT	PK	
id_usuario	INT	FK (usuarios)	Usuario al que se le aplica
id_recurso	INT	FK (recursos)	Recurso al que se le aplica
accion	VARCHAR (10)	NOT NULL	allow o deny
expiration	TIMESTAMP	NOT NULL	Tiempo en el que expira el permiso

Ante cualquier modificación se emplea un bundle (servicio que genera paquete de datos .tar) para refrescar las políticas haciendo un polling cada 60 seg, esto permite actualizar las nuevas políticas desde la DB sin necesidad de reiniciar OPA

(También se puede implementar una tabla de excepciones para multirol pero se debería añadir un bloque Rego de excepciones y sincronizar la nueva tabla con Bundle y que M6 verifique la sesión activa e instale la regla en T3)

Con el input y la data OPA en M2 devolverá al módulo de traducción M6 la lista de permisos (allow/deny) de acuerdo al recurso. En este se indican la IP destino, puerto, protocolo, tabla OpenFlow y el tiempo de vida.

Funcionamiento de OPA:

Después de que M6 recibe el Token de rol de M1, este crea un JSON y lo envía mediante un POST al puerto 8181 directamente a OPA

```
{
  "input": {
    "codigo_pucp": "20200123",
    "rol": "Estudiante_Telecom",
    "ip_asignada": "10.2.1.45",
    "cidr_rol": "10.2.1.0/24",
    "mac_address": "aa:bb:cc:dd:ee:ff",
    "switch_dpid": "of:0000000000000001",
    "in_port": 3,
    "hora": "2026-06-02T10:30:00Z"
  }
}
```

OPA recibe el POST y ejecuta el archivo de Policy REGO quien se encarga de leer el input y buscar lo que se encuentra en el campo "rol" e IP, una vez identificado busca en su data si existen reglas para el usuario y si la IP no forma parte de una lista negra.

```
{
  "result": {
    "allow": true,
    "permisos": [
      {
        "recurso": "cursos_telecom",
        "ip_dst": "10.0.0.21",
        "puertos": [80, 443],
        "protocolo": "tcp",
        "tabla": "T2",
        "prioridad": 100,
        "timeout": null,
        "accion": "ALLOW"
      }
    ]
  },
}
```

```

"denegaciones": [
  {
    "ip_dst": "10.0.0.22",
    "puertos": [80, 443],
    "tabla": "T3",
    "prioridad": 200,
    "accion": "DROP"
  },
  {
    "ip_dst": "10.0.0.23",
    "puertos": [80, 443],
    "tabla": "T3",
    "prioridad": 200,
    "accion": "DROP"
  },
  {
    "ip_dst": "10.0.0.30",
    "puertos": [80, 443],
    "tabla": "T3",
    "prioridad": 200,
    "accion": "DROP"
  }
],

```

M6 recibe las instrucciones de OPA y traduce estas en OpenFlow principalmente a las tablas T2 (autorización por IP) y T3 (denegaciones)

Tabla 2: Acceso de estudiante de Telecomunicaciones a sus cursos

- Prioridad
100
- nw_src
10.2.1.0/24
- nw_dst
10.0.0.21
- nw_proto
6 (TCP)
- tp_dst
80 / 443
- Acción
output: puerto_servidor_2

Tabla 3: Reglas DROP con mayor prioridad que las de T2

- Prioridad
200

- l dl_src (MAC)
- aa:bb:cc:dd:ee:ff
- nw_src
- 10.2.1.45
- nw_dst
- 10.0.0.22
- tp_dst
- 80 / 443
- Acción
- DROP
- hard_timeout
- duración sesión (seg)

Implementación de DEMO

- Roles y políticas de autorización

Host	IP	Rol	Puerto servidor
H1	10.0.0.1	student	TCP 80, 443
H2	10.0.0.2	teacher	TCP 80, 443, 8080
H3	10.0.0.3	adminTi	TCP 80, 443, 8080, 22
H4	10.0.0.4	server	recibe todo

Origen Student to Destino Server (Tipo de intents: PointToPointIntent)

- TCP 80,443: servidor de cursos
- ICMP, SSH, 8080 hace DROP

Origen Teacher to Destino Server (Tipo de intents: PointToPointIntent)

- TCP 80, 443 = servidor de cursos
- TCP 8080 = servidor de notas
- ICMP, SSH hace DROP

Origen AdminTi to Destino Server (Tipo de intents: PointToPointIntent)

- Protocolo TCP y puertos (80, 443, 8080, 22)

Student ping Server

```

root@sw2:~# sudo ovs-ofctl -O OpenFlow13 dump-flows sw2
cookie=0x10000d1d72504, duration=44707.680s, table=0, n_packets=14424, n_bytes=1961664, send_flow_rem priority=40000,dl_type=0x8942 actions=CONTROLLER:65535,clear_actions
cookie=0x100003afe2839, duration=44707.680s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=40000,dl_type=0x88cc actions=CONTROLLER:65535,clear_actions
cookie=0x10000e388d4f8, duration=44707.605s, table=0, n_packets=11, n_bytes=552, send_flow_rem priority=40000,arp actions=CONTROLLER:65535,clear_actions
cookie=0x10000355550f1, duration=44010.812s, table=0, n_packets=8, n_bytes=736, send_flow_rem priority=5,ip actions=CONTROLLER:65535,clear_actions
cookie=0x0d00002e491694, duration=6.030s, table=0, n_packets=3, n_bytes=204, send_flow_rem priority=40000,icmp,dl_src=fa:16:3e:14:78:63,dl_dst=fa:16:3e:29:7c:47,nw_src=10.0.0.1,nw_dst=10.0.0.4 actions=drop
cookie=0xac0000ab255fe1, duration=180.806s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=100,tcp,in_port=ens5,nw_src=10.0.0.1,nw_dst=10.0.0.4,tp_dst=80 actions=output:ens4
cookie=0xac00007bb2dbda, duration=180.701s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=100,tcp,in_port=ens5,nw_src=10.0.0.1,nw_dst=10.0.0.4,tp_dst=443 actions=output:ens4
cookie=0xac0000b2f23a84, duration=180.787s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=100,tcp,in_port=ens4,nw_src=10.0.0.4,nw_dst=10.0.0.1,tp_src=80 actions=output:ens5
cookie=0xac0000fe576f16, duration=180.705s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=100,tcp,in_port=ens4,nw_src=10.0.0.4,nw_dst=10.0.0.1,tp_src=443 actions=output:ens5
root@sw2:~#

```

```

rbac>
rbac> users

  NAME      ROLE      IP      MAC      STATE
  -----
  student   student   10.0.0.1  FA:16:3E:14:78:63  * Connected
  teacher    teacher   10.0.0.2  FA:16:3E:E9:BF:92   Disconnected
  admin_ti   admin_ti   10.0.0.3  FA:16:3E:F1:78:25   Disconnected
  server     server     10.0.0.4  FA:16:3E:29:7C:47   * Connected

* = Connected (active in policy)
rbac>

```

Implementación de FreeRADIUS

Archivo de MySQL: *radius_db_pucp_sdn.sql*

Se usó una VM con Ubuntu en VirtualBox para que MySQL y FreeRADIUS compartan la misma red interna y no haya problemas de conectividad entre ambos servicios.

1. Instalación de FreeRADIUS

En la terminal se ingresan los siguientes comandos:

```

sudo apt update
sudo apt install freeradius freeradius-mysql -y

```

Se instalan dos paquetes: **freeradius**, que es el servidor principal de autenticación, y **freeradius-mysql**, que es el módulo de enlace (**rlm_sql_mysql**) compatible con MySQL. Este segundo paquete añade las librerías dinámicas necesarias para que el servidor pueda conectarse a un motor de base de datos relacional mediante consultas SQL.

Para verificar que MySQL se instaló y está en ejecución se usa el siguiente comando:

```

sudo systemctl status mysql

```

Resultado obtenido:

```
sheila@sheila-VirtualBox:~$ sudo systemctl status mysql
[sudo] contraseña para sheila:
● mysql.service - MySQL Community Server
   Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-06-07 03:07:56 -05; 16h ago
     Main PID: 5818 (mysqld)
    Status: "Server is operational"
      Tasks: 46 (limit: 4584)
     Memory: 53.0M
    CGroup: /system.slice/mysql.service
            └─5818 /usr/sbin/mysqld

jun 07 03:07:54 sheila-VirtualBox systemd[1]: Starting MySQL Community Server...
jun 07 03:07:56 sheila-VirtualBox systemd[1]: Started MySQL Community Server.
sheila@sheila-VirtualBox:~$
sheila@sheila-VirtualBox:~$
```

Una vez creada la base de datos, se importa el script SQL que contiene la estructura y los datos del proyecto. Desde una PC con Windows, se transfiere el archivo a la VM mediante SCP:

```
scp radius_db_pucp_sdn.sql [usuario_VM]@[ip_VM]:/home/sheila/      #Indicar la ruta en
donde se quiere guardar
```

Una vez transferido, se importa desde la terminal de la VM:

```
sudo mysql radius_db < radius_db_pucp_sdn.sql
```

Se verifica que todas las tablas fueron creadas correctamente con el siguiente comando:

```
sudo mysql radius_db -e "SHOW TABLES;"
```

Las tablas fueron importadas de forma correcta:

```

sheila@sheila-VirtualBox:~/Descargas$ sudo mysql radius_db -e "SHOW TABLES;"
+-----+
| Tables_in_radius_db |
+-----+
| lista_negra_t0      |
| politicas_rbac      |
| politicas_temporales |
| radacct              |
| radcheck             |
| radgroupcheck        |
| radgroupreply        |
| radreply            |
| radusergroup         |
| recursos             |
| roles_facultad       |
| servidores          |
| sesiones_activas     |
| usuarios            |
| usuarios_rols        |
+-----+
sheila@sheila-VirtualBox:~/Descargas$

```

2. Creación de usuario en MySQL

FreeRADIUS no debe conectarse con la cuenta root por razones de seguridad, por lo que se crea una cuenta exclusiva con permisos mínimos:

```

CREATE USER 'radius'@'localhost' IDENTIFIED BY 'radius_pass';
GRANT SELECT ON radius_db.radcheck TO 'radius'@'localhost';
GRANT SELECT ON radius_db.radreply TO 'radius'@'localhost';
GRANT SELECT ON radius_db.radusergroup TO 'radius'@'localhost';
GRANT SELECT ON radius_db.radgroupcheck TO 'radius'@'localhost';
GRANT SELECT ON radius_db.radgroupreply TO 'radius'@'localhost';
GRANT ALL ON radius_db.radacct TO 'radius'@'localhost';
GRANT ALL ON radius_db.radpostauth TO 'radius'@'localhost';
GRANT SELECT ON radius_db.usuarios TO 'radius'@'localhost';
FLUSH PRIVILEGES;

```

Los permisos se asignan de forma restrictiva: SELECT en las tablas de lectura (credenciales, grupos y respuestas) y ALL únicamente en las tablas donde FreeRADIUS necesita escribir, que son radacct para el registro de sesiones y radpostauth para el log de autenticaciones. Se utiliza 'localhost' porque tanto FreeRADIUS como MySQL residen en la misma VM de VirtualBox, lo que restringe cualquier intento de conexión externa a este usuario. En caso de que FreeRADIUS y MySQL se encuentren en máquinas distintas, se debe reemplazar 'localhost' por la IP del servidor donde corre MySQL y asegurarse de que el puerto 3306 sea accesible desde la red

3. Configuración del Driver SQL en FreeRADIUS

Se modifica el archivo principal de configuración del módulo de base de datos de FreeRADIUS para indicarle los parámetros de red y las credenciales de acceso a MySQL:

```
sudo nano /etc/freeradius/3.0/mods-available/sql
```

Parámetros editados:

```
driver = "rlm_sql_mysql"
dialect = "mysql"
server = "localhost"
port = 3306
login = "radius"
password = "radius_pass"
radius_db = "radius_db"
```

```
#
dialect = "mysql"

#
# The driver module used to execute the queries. Since we
# don't know which SQL drivers are being used, the default is
# "rlm_sql_null", which just logs the queries to disk via the
# "logfile" directive, below.
#
# In order to talk to a real database, delete the next line,
# and uncomment the one after it.
#
# If the dialect is "mssql", then the driver should be set to
# one of the following values, depending on your system:
#
#     rlm_sql_db2
#     rlm_sql_firebird
#     rlm_sql_freetds
#     rlm_sql_iodbc
#     rlm_sql_unixodbc
#     rlm_sql_mongo
#
driver = "rlm_sql_mysql"
driver = "rlm_sql_${dialect}"

#
```

```

# Connection info:
#
server = "localhost"
port = 3306
login = "radius"
password = "radius_pass"

# Connection info for Mongo
# Authentication Without SSL
#     server = "mongodb://USER:PASSWORD@192.16.0.2:PORT/DATABASE?authSource=admin&ssl=false"

# Authentication With SSL
#     server = "mongodb://USER:PASSWORD@192.16.0.2:PORT/DATABASE?authSource=admin&ssl=true"

# Authentication with Certificate
# Use this command for retrieve Derived username:
# openssl x509 -in mycert.pem -inform PEM -subject -nameopt RFC2253
# server = mongodb://<DERIVED USERNAME>@192.168.0.2:PORT/DATABASE?authSource=$external&ssl=true"

# Database table configuration for everything except Oracle
radius_db = "radius_db"

```

Observación: Durante la configuración se presentó un error al ejecutar el modo debug (sudo freeradius -X) relacionado con el bloque tls dentro de la sección mysql del archivo. El error indicaba Failed parsing configuration item "ca_file". La solución fue comentar todas las líneas dentro del bloque tls, ya que la conexión entre FreeRADIUS y MySQL es local y no requiere cifrado TLS.

```

mysql {
    # If any of the files below are set, TLS encryption is enabled
    tls {
        # ca_file = "/etc/ssl/certs/my_ca.crt"
        # ca_path = "/etc/ssl/certs/"
        # certificate_file = "/etc/ssl/certs/private/client.crt"
        # private_key_file = "/etc/ssl/certs/private/client.key"
        # cipher = "DHE-RSA-AES256-SHA:AES128-SHA"

        # tls_required = yes
        # tls_check_cert = no
        # tls_check_cert_cn = no
    }

    # If yes, (or auto and libmysqlclient reports warnings are
    # available), will retrieve and log additional warnings from
    # the server if an error has occurred. Defaults to 'auto'
    warnings = auto
}

```

4. Activar el módulo SQL

Se debe cargar los módulos que tienen enlace simbólico en la carpeta mods-enabled. Para activarlo se ejecuta:

```

sudo ln -s /etc/freeradius/3.0/mods-available/sql \
    /etc/freeradius/3.0/mods-enabled/sql

```

5. Testeo desde terminal

Ejecutamos en el terminal 1 el modo debug de FreeRADIUS para comprobar la conexión con MySQL

```
sudo freeradius -X
```

En el terminal 2 comprobamos la conexión a la db y si devuelve un Access accept o Access Reject

```
radtest 20192434 pass_teleco123 localhost 0 testing123
```

```
sheila@sheila-VirtualBox:~$ radtest 20192434 password_teleco localhost 0 testing123
Sent Access-Request Id 158 from 0.0.0.0:46167 to 127.0.0.1:1812 length 78
  User-Name = "20192434"
  User-Password = "password_teleco"
  NAS-IP-Address = 127.0.1.1
  NAS-Port = 0
  Cleartext-Password = "password_teleco"
Received Access-Accept Id 158 from 127.0.0.1:1812 to 127.0.0.1:46167 length 58
  Message-Authenticator = 0x5194a7d7fc2924e96b7809ea9a091256
  Filter-Id = "Estudiante_Telecom"
sheila@sheila-VirtualBox:~$
```

Tablas de FreeRADIUS

FreeRADIUS tiene tablas nativas que permite leer de forma directa ya que tiene un esquema fijo y no se puede renombrar:

- radcheck: Credenciales de usuarios

Es la tabla más importante. Cuando llega un Access-Request, FreeRADIUS busca aquí la contraseña del usuario y la compara con la que viene en el paquete. Si coinciden, la autenticación avanza; si no, responde con Access-Reject.

Tiene cuatro campos:

- username: el código PUCP del usuario
- attribute: el tipo de verificación, en este caso Cleartext-Password
- op: el operador de comparación, := significa "asignar y verificar"
- value: la contraseña del usuario

- radusergroup: Asignación usuario a grupo

Una vez que FreeRADIUS verificó la contraseña en radcheck, consulta esta tabla para saber a qué grupo pertenece el usuario. El grupo determina qué atributos se van a devolver en el Access-Accept.

Tiene tres campos:

- username: el código PUCP

- groupname: el nombre del grupo, por ejemplo Estudiante_Telecom
- priority: si un usuario pertenece a varios grupos, se usa el de mayor prioridad

- **radgroupreply: Atributos de respuesta por grupo**

Aquí FreeRADIUS busca qué atributos incluir en el Access-Accept para cada grupo. El atributo más importante es Filter-Id, que lleva el nombre del rol hacia el Portal Cautivo. También se define el Session-Timeout, que indica el tiempo máximo de sesión en segundos.

Por ejemplo, el grupo Estudiante_Telecom devuelve:

- Filter-Id = "Estudiante_Telecom" → el portal sabe el rol
- Session-Timeout = 28800 → 8 horas de sesión

- **radreply: Atributos individuales por usuario**

Similar a radgroupreply pero a nivel de usuario específico. Si un usuario necesita un atributo diferente al del grupo, se define aquí. En el proyecto esta tabla está vacía porque el rol siempre viene del grupo.

- **radacct: Accounting de sesiones**

FreeRADIUS escribe aquí automáticamente cuando recibe mensajes de accounting del Portal Cautivo: Accounting-Request Start al inicio de sesión, Update durante la sesión y Stop al cerrarla. No requiere inserción manual. El campo callingstationid almacena la MAC del dispositivo cliente.

- **radpostauth: Log de autenticaciones**

FreeRADIUS registra aquí cada autenticación con su resultado: Access-Accept o Access-Reject, junto con el timestamp. Es útil para la auditoría del sistema.

Instalación de DHCP

Testeo Final para Flujo de Autenticación completo